MAE 162E Capstone Lab

Week 5 – Obstacle Avoidance with 2-D LiDAR

Lab Objectives

Obstacle avoidance is an important topic for autonomous driving. By the end of this lab, students should be able to:

- Capture 2-D **LiDAR** information based on ROS2-based system.
- Understand and implement the obstacle avoidance approach that **generates collision-free waypoints** in real-time from LiDAR feedback and integrates with a **Pure Pursuit** controller to produce linear and angular velocity commands.
- Analyze and tune key algorithm parameters, and explain how each parameter influences the robot's motion, stability, and safety.
- Evaluate the limitations of the implemented approach and propose improvements or alternative algorithms to enhance robustness and generalization beyond controlled environments.

Introduction of Pure Pursuit with Obstacle Avoidance

2D LiDAR is widely used in autonomous driving due to several key advantages:

1. Accurate perception: LiDAR directly measures distances to surrounding objects, providing reliable depth information compared to vision-based methods that must infer depth.
2. High spatial resolution: LiDAR enables the detection of small obstacles and captures detailed geometric features of the environment.
3. Lighting independence: LiDAR operates effectively in low-light, shadowed, and high-glare environments where cameras may struggle.

A variety of real-time obstacle avoidance methods have been developed, such as the Dynamic Window Approach (DWA), Artificial Potential Field (APF), and Reinforcement Learning (RL). However, in this class, the driving environment is relatively structured (e.g., a two-lane track) and does not require highly complex planning strategies. Therefore, we adopt a simpler and more intuitive approach: combining obstacle avoidance with a Pure Pursuit path-tracking controller. This

method leverages LiDAR data to adjust waypoints in real-time, enabling the robot to follow a desired path while avoiding collisions.

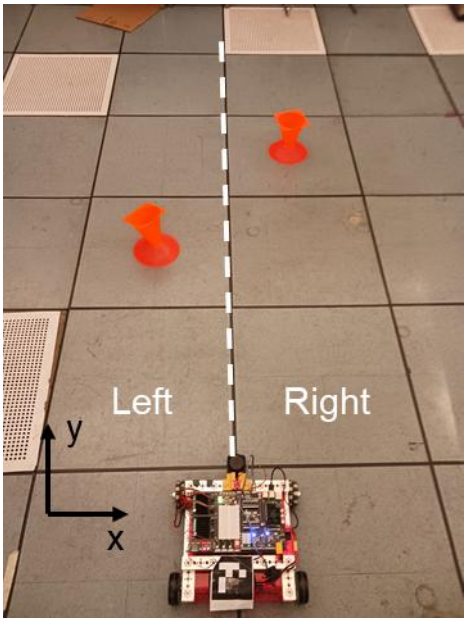

Figure 1: There are only 2 lanes in our environment with the center of left and right lanes (centerline) at x_L parallel to the y-axis of the world (ground) frame, which origin is the starting point.

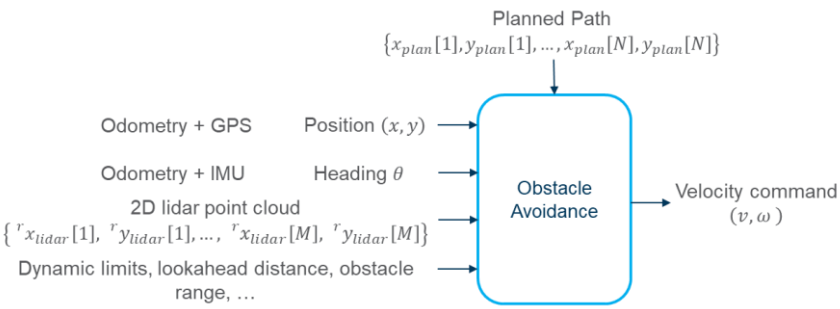

Figure 2: Input and output parameters of dynamic window approach.

To perform obstacle avoidance, the robot must first have access to a set of pre-planned path waypoints. During each control loop, the robot receives its current pose from onboard sensors such as wheel encoders (odometry) and GPS. Using this pose information together with LiDAR point cloud data, updated waypoints are generated based on a set of tunable parameters.

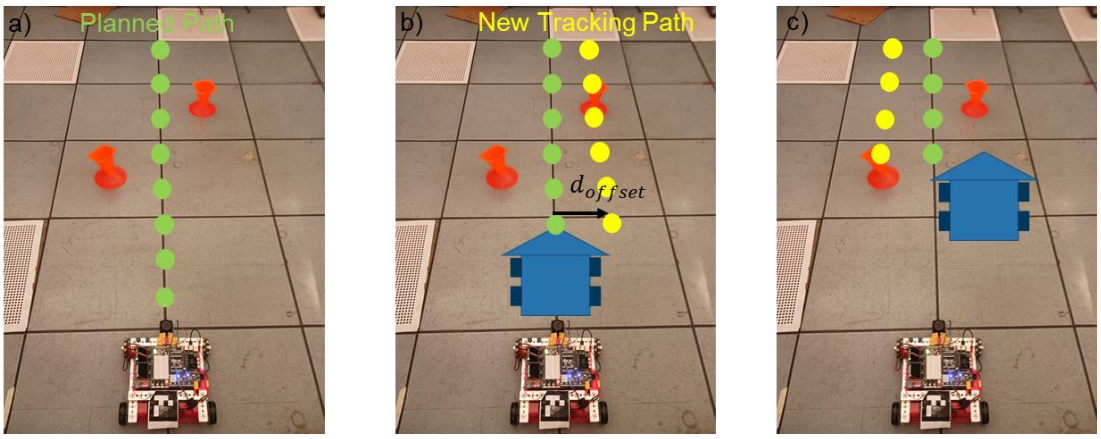

Figure 3: Basic concept of obstacle avoidance approach.

The core idea of this approach is to determine whether an obstacle lies in the left or right lane and then apply a lateral offset to the original path accordingly. This offset produces a new tracking path, which is followed by the Pure Pursuit controller to generate real-time velocity commands.

For example, the robot initially follows the planned path, as shown in Fig. 3(a). When an obstacle is detected in the left lane, a rightward offset is applied to the path, resulting in a new trajectory that guides the robot into the right lane, as illustrated in Fig. 3(b). Conversely, when an obstacle is detected in the right lane, a leftward offset is introduced, causing the robot to follow a path in the left lane, as shown in Fig. 3(c).

When the planned path $\{x_{plan}[k], y_{plan}[k]\}$ is known, the obstacle avoidance procedure can be described as follows:

1. Obtain current state: The current state includes the robot's pose (x, y, θ) and the obstacle point cloud expressed in the robot frame $\{{}^{r}x_{obstacle}[k],\ {}^{r}y_{obstacle}[k]\}$. The LiDAR point cloud must first be transformed according to the sensor's mounting configuration. For example, on the TAs' robot, a $180°$ rotation is required to align the LiDAR frame with the robot frame. After transformation, the point cloud is filtered based on a specified field-of-view angle and obstacle detection range. This filtering step limits the search region to a circular-sector-shaped area in front of the robot, reducing computational load and focusing on the most relevant obstacles for real-time avoidance.

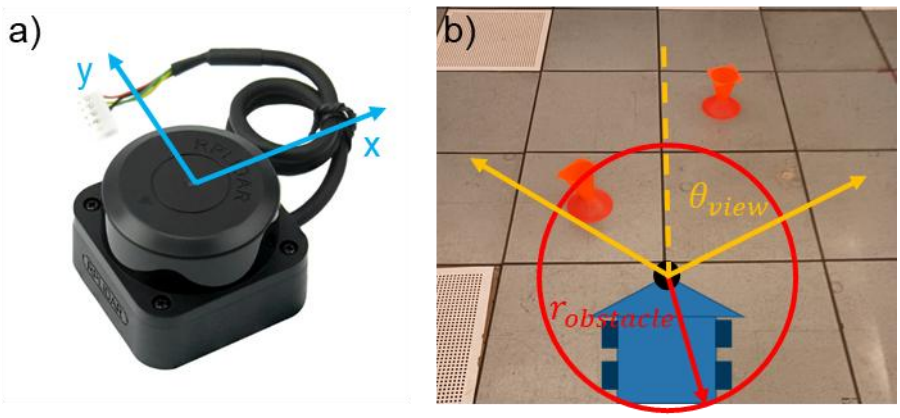

Figure 4: a) The lidar point cloud coordinate provided by the vender. b) The searching region of obstacles will be a circular sector in front of the robot.

2. Obstacle filtering and path modification: In this step, the obstacle point cloud is further filtered using prior knowledge of the lane geometry, including the centerline location x_L and lane width l_{lane}. This allows the system to focus on obstacles that are relevant to the current driving lane. Next, each planned waypoint is evaluated with respect to nearby obstacles. If the distance between a waypoint and any obstacle is smaller than a predefined safety threshold d_{safe}, the waypoint is considered unsafe. In such cases, the waypoint is adjusted to a new position that shifts it away from the obstacle while maintaining path continuity. Typically, the modified waypoint is placed between the original waypoint and the subsequent waypoint.

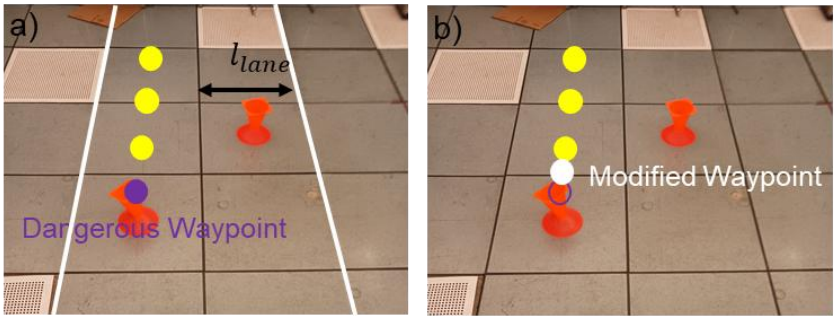

Figure 5: Path modification.

Note that the lane boundaries and planned path waypoints are defined in the **world frame**, which is fixed to the ground. In contrast, the LiDAR sensor is mounted on the robot, so the obstacle point cloud $\{{}^{r}x_{obstacle}[k], {}^{r}y_{obstacle}[k]\}$ is naturally expressed in the **robot frame** and changes as the robot (x, y, θ) moves. To enable consistent comparison between obstacles and the planned path, the obstacle points must be transformed from the robot frame to the world frame using the robot's current pose (position and orientation). This transformation ensures that both the path waypoints and

obstacle data are represented in the same coordinate frame for accurate distance evaluation and path modification.

$$\begin{bmatrix} x_{lidar} \\ y_{lidar} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} {}^{r}x_{lidar} \\ {}^{r}y_{lidar} \end{bmatrix} + \begin{bmatrix} x \\ y \end{bmatrix}$$

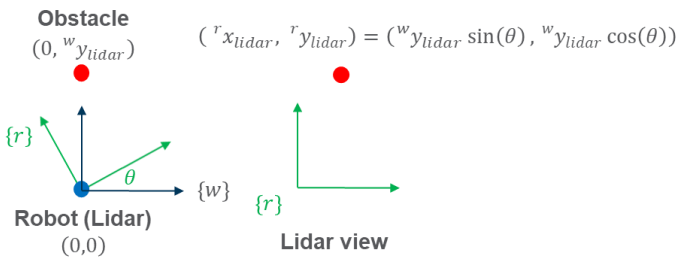

Figure 6: This example shows the difference between the obstacle position viewed from world and robot coordinate.

3. Waypoints generation: In this approach, only the closest obstacle point detected by the LiDAR is considered for decision-making. If this obstacle lies on the left side of the centerline, the new waypoints $\{x_{path}[k], y_{path}[k]\}$ will shift from the planned path (on the centerline) to the right with a certain offset d_{offset}. Conversely, if the obstacle is on the right side, the path is shifted to the left. To prevent frequent and unstable path updates, the system introduces an **avoidance delay**: once a new path is generated, further updates are temporarily suspended for a fixed number of control loops. This allows the robot to commit to the current avoidance maneuver without continuously changing its target trajectory. Additionally, to improve tracking of the newly generated path, the lookahead distance in the Pure Pursuit controller can be temporarily reduced by applying a scaling factor $\alpha_{L_d} < 1$.

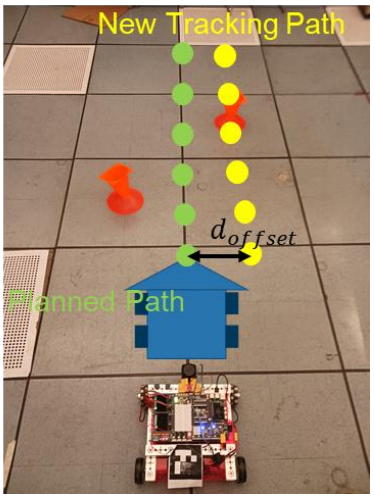

Figure 7: Waypoint generation.

Finally, the current target point (x_{target}, y_{target}) is determined from the active

tracking path $\{x_{path}[k], y_{path}[k]\}$ using the lookahead distance L_d, following the same procedure introduced in the Pure Pursuit algorithm in Week 2. Based on this target point, along with the maximum allowable linear velocity v_{max} and angular velocity ω_{max}, the controller computes the corresponding velocity commands for the robot. The algorithm continues to update and execute these commands until the robot reaches the final waypoint within a specified goal tolerance $d_{tolerance}$, at which point the motion is terminated.

How to Improve?

- Currently, the obstacle detection region is defined as a circular-sector shape in front of the robot. Can you design alternative shapes that may better suit different scenarios? For example, a rectangular region aligned with the lane, an adaptive ellipse based on speed, or a dynamically adjusted sector based on steering direction.
- Due to the limited field of view, the robot may "forget" previously detected obstacles once they leave the sensing range, increasing the risk of collision. One possible improvement is to introduce a short-term memory (buffer) that retains obstacle information for a certain duration, allowing the robot to maintain awareness of recently observed obstacles.
- More sophisticated obstacle avoidance approaches can also be explored, such as the Dynamic Window Approach (DWA), Artificial Potential Field (APF), and Reinforcement Learning (RL). These methods may provide improved performance in more complex and dynamic environments, but often require additional computation, tuning, or training.

Task 1: LiDAR Point Cloud Acquisition

1. **Under your project folder**, use the following code to pull code from the TA's GitHub repository. Carefully merge the code if you have already made your own modifications. If you haven't added the TA's repo as your upstream, please take reference of Week 1 lab to do so.

```
git fetch upstream
git merge upstream/main
```

2. Use the following code to initialize the docker on your Raspberry PI.

```
COMPOSE=ros2_ws/docker/docker-compose.rpi.yml
docker compose -f $COMPOSE build
docker compose -f $COMPOSE up -d
docker compose -f $COMPOSE logs -f ros2_runtime
```

3. Run lidar node inside docker. Once the node starts, you should hear the sound of the moving motor inside your 2D lidar.

```
COMPOSE=ros2_ws/docker/docker-compose.rpi.yml
docker compose -f $COMPOSE exec ros2_runtime bash
```

```
source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
ros2 run rplidar_ros rplidar_node --ros-args -p channel_type:="serial" -
p serial_port:="/dev/rplidar" -p serial_baudrate:=460800 -p
frame_id:="laser_frame" -p angle_compensate:=true -p
scan_mode:="Standard"
```

4. **In another terminal**, open a new docker, and run the robot launch file which runs both bridge and robot nodes in docker.

```
COMPOSE=ros2_ws/docker/docker-compose.rpi.yml
```

```
docker compose -f $COMPOSE exec ros2_runtime bash
```

```
source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash

ros2 launch robot robot.launch.py
```

5. **In the third terminal**, **don't** enter the docker. Copy the point cloud PNG file from docker to your project folder.

```
docker cp docker-ros2_runtime-1:/data/plot.png .
```

6. Finally, you should see a "plot.png" file under your project folder, which is similar to Fig. 8.

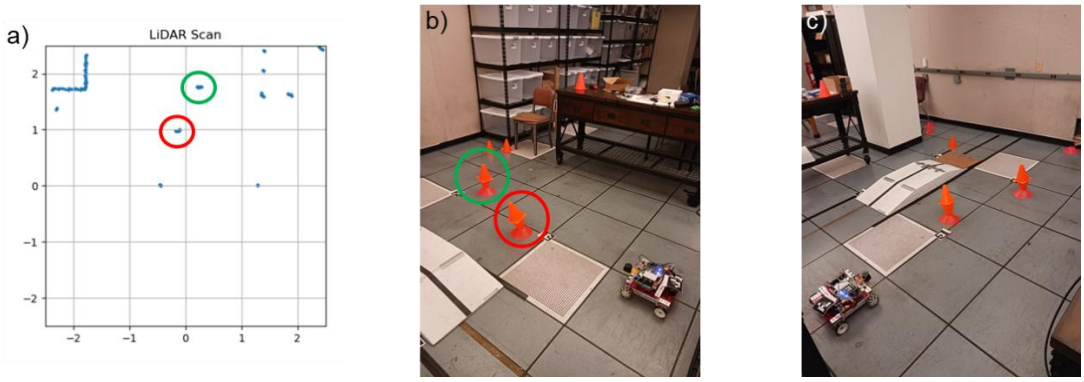

Figure 8: a) LiDAR scanning result. b) c) The view from the robot.

Task 2: Implementation of Obstacle Avoidance Approach

1. Change the transformation based on the way you mount your LiDAR in **Project_NUEVO/ros2_ws/src/robot/robot/path_planner.py**

```
if len(obstacles_r) > 0:
    # lidar orientation due to installation is 180 deg rotated from robot forward, so rotate obstacles accordingly.
    obstacles_r = (np.array([[np.cos(np.pi), -np.sin(np.pi)], [np.sin(np.pi), np.cos(np.pi)]]) @ obstacles_r.T).T
```

Figure 9: Based on how the LiDAR is mounted, the coordinate should be transformed.

2. Tune all parameters set for the robot in **Project_NUEVO/ros2_ws/src/robot/robot/main.py**

What should be carefully designed?

- Planned path $\{x_{plan}[k], y_{plan}[k]\}$: The location of each waypoint will highly affect the performance of pure pursuit.
- The location of centerline x_L should be carefully measured.

What are the tunable parameters?

- Lookahead distance L_d
- Maximal linear speed v_{max}
- Maximal angular speed ω_{max}
- Goal tolerance $d_{tolerance}$
- Obstacle range $r_{obstacle}$
- View angle θ_{view}
- Safe distance d_{safe}
- Avoidance delay: Once a new path for avoidance is generated, further updates are temporarily suspended for a fixed number of control loops.
- Offset d_{offset}
- Lane width l_{lane}
- Lookahead distance factor $\alpha_{L_d} < 1$: To improve tracking of the newly generated path, the lookahead distance in the Pure Pursuit controller can be temporarily reduced by applying a scaling factor.

3. Repeat 1 to 4 of Task 1 until the robot can run through the obstacles on the left and/or right lane.

```
robot._nav_follow_pp_path(
    lookahead_distance=100.0,
    max_linear_speed=140.0,
    max_angular_speed=1.5,
    goal_tolerance=20.0,
    obstacles_range=450.0,
    view_angle=math.radians(70.0),
    safe_dist=250.0,
    avoidance_delay=150,
    alpha_Ld=0.7,
    offset=270.0,
    lane_width=500.0,
    obstacle_avoidance=True,
    x_L=300.0,
)
```

Figure 10: Tunable parameters.